CM3020 Part B: Combined Source Code

Highlighted lines show differences relative to the provided starter codebase.

All Python source files from my codebase have been concatenated in this document. For files that exist in the starter codebase, the document highlights only the lines that differ from the starter version. For new scripts (experiments, plots, summaries, and demos), the entire file code is highlighted.

Files included

File	Comparison with starter
creature.py	Modified relative to starter
cw-envt.py	Matches starter
demo_gui.py	New file (not in starter)
demo_mountain_gui.py	New file (not in starter)
genome.py	Modified relative to starter
motor_test.py	Matches starter
offline_from_csv.py	Matches starter
plot_air_ratio.py	New file (not in starter)
plot_runs.py	New file (not in starter)
population.py	Modified relative to starter
prepare_shapes.py	Matches starter
realtime_from_csv.py	Matches starter
run_ga_mountain.py	New file (not in starter)
simulation.py	Modified relative to starter
starter.py	Matches starter
summarise_grouped.py	New file (not in starter)
summarise_grouped_conditions.py	New file (not in starter)
summarise_runs.py	New file (not in starter)

test_creature.py	Matches starter
test_ga.py	Matches starter
test_ga_no_threads.py	Modified relative to starter
test_genome.py	Matches starter
test_population.py	Matches starter
test_simulation.py	Matches starter

simulation.py

File: simulation.py

Highlighted lines indicate differences relative to the starter codebase.

```
import pybullet as p
from multiprocessing import Pool

# Added: load the mountain URDF and clean up temporary URDF files
import os
import time
import pybullet_data

"""
simulation.py

Extends the starter simulation to support the mountain climbing task.
Key additions:
- Mountain arena construction and mountain peak estimation
- Fitness metrics that reward climbing and penalise flying or long airtime
- Optional GUI mode used only for the demo video

The experiment pipeline continues to use DIRECT mode for speed and consistency.
"""

# Experiments use DIRECT for speed. GUI is reserved for video demo only.
class Simulation:
    def __init__(self, sim_id=0, connection_mode="DIRECT"):
        """
        connection_mode: "DIRECT" (fast, default for experiments) or "GUI" (for demo video)
        Backward compatible: Simulation() and Simulation(i) still work.
        """
        self.sim_id = sim_id

        mode = p.DIRECT
        if isinstance(connection_mode, str) and connection_mode.upper() == "GUI":
            mode = p.GUI

        self.physicsClientId = p.connect(mode)

        # Optional GUI cosmetics only when GUI is used
        if mode == p.GUI:
            try:
                p.configureDebugVisualizer(p.COV_ENABLE_GUI, 0, physicsClientId=self.physicsClientId)
            except Exception:
                pass
```

```
    # Added: build the mountain evaluation world in DIRECT mode.
    # Builds the arena and loads the gaussian_pyramid URDF.
    # Collision-only geometry keeps DIRECT runs fast.
    def build_mountain_world(self, pid, arena_size=20, wall_height=1):
        # Reset and basic physics setup should already be done before calling this
        p.setAdditionalSearchPath(pybullet_data.getDataPath(), physicsClientId=pid)

        shapes_path = os.path.join(os.path.dirname(__file__), "shapes")
        p.setAdditionalSearchPath(shapes_path, physicsClientId=pid)

        # Arena: floor and walls (collision only is enough for DIRECT)
        wall_thickness = 0.5

        floor_col = p.createCollisionShape(
            shapeType=p.GEOM_BOX,
            halfExtents=[arena_size/2, arena_size/2, wall_thickness],
            physicsClientId=pid
        )
        floor_id = p.createMultiBody(
            baseMass=0,
            baseCollisionShapeIndex=floor_col,
            basePosition=[0, 0, -wall_thickness],
            physicsClientId=pid
        )

        wall_col_x = p.createCollisionShape(
            shapeType=p.GEOM_BOX,
            halfExtents=[arena_size/2, wall_thickness/2, wall_height/2],
            physicsClientId=pid
        )
        p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_col_x,
                          basePosition=[0, arena_size/2, wall_height/2], physicsClientId=pid)
        p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_col_x,
                          basePosition=[0, -arena_size/2, wall_height/2], physicsClientId=pid)

        wall_col_y = p.createCollisionShape(
            shapeType=p.GEOM_BOX,
            halfExtents=[wall_thickness/2, arena_size/2, wall_height/2],
            physicsClientId=pid
        )
        p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_col_y,
                          basePosition=[arena_size/2, 0, wall_height/2], physicsClientId=pid)
        p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_col_y,
                          basePosition=[-arena_size/2, 0, wall_height/2], physicsClientId=pid)

        # Load the prepared mountain URDF and build arena walls to keep the creature inside the
sandbox.
        mountain_position = (0, 0, -1)
        mountain_orientation = p.getQuaternionFromEuler((0, 0, 0))
        mountain_id = p.loadURDF(
            "gaussian_pyramid.urdf",
```

```
            mountain_position,
            mountain_orientation,
            useFixedBase=1,
            physicsClientId=pid
        )

        return {"floor_id": floor_id, "mountain_id": mountain_id}

    # Added: estimate the mountain peak using the URDF AABB.
    # Used for distance-to-peak tracking in fitness.
    def get_mountain_peak(self, mountain_id, pid):
        aabb_min, aabb_max = p.getAABB(mountain_id, physicsClientId=pid)
        peak_x = 0.5 * (aabb_min[0] + aabb_max[0])
        peak_y = 0.5 * (aabb_min[1] + aabb_max[1])
        peak_z = aabb_max[2]
        return (peak_x, peak_y, peak_z)

    # Changed: evaluation now runs on the mountain arena,
    # so fitness reflects climbing progress.
    def run_creature(
        self,
        cr,
        iterations=2400,
        start_pos=(0, 0, 8),
        control_period=24,
        motor_force=5,
        settle_steps=24,
        realtime=False,
        sleep_dt=1/240,
        # Demo-only assists (default False so experiments are unaffected)
        demo_mode=False,
        demo_link_friction=2.0,
        demo_env_friction=2.0,
        demo_linear_damping=0.0,
        demo_angular_damping=0.0,
        demo_solver_iters=150,
        demo_vel_scale=8.0
    ):

        pid = self.physicsClientId

        # Reset simulation for this evaluation
        p.resetSimulation(physicsClientId=pid)
        p.setPhysicsEngineParameter(enableFileCaching=0, physicsClientId=pid)
        p.setGravity(0, 0, -10, physicsClientId=pid)

        # Build arena + mountain
        world_ids = self.build_mountain_world(pid, arena_size=20)
        mountain_id = world_ids["mountain_id"]
        floor_id = world_ids["floor_id"]
```

```
        # Peak point from mountain AABB
        peak = self.get_mountain_peak(mountain_id, pid)

        # Write temporary URDF and load creature
        xml_file = "temp" + str(self.sim_id) + ".urdf"
        xml_str = cr.to_xml()
        with open(xml_file, "w") as f:
            f.write(xml_str)

        cid = p.loadURDF(xml_file, physicsClientId=pid, basePosition=list(start_pos))

        # Remove temp URDF so long runs do not clutter the folder
        try:
            os.remove(xml_file)
        except OSError:
            pass
        # Demo-only physics assists (only used for the video).
        # Guarded by demo_mode so experiments stay unchanged.
        if demo_mode:
            # More stable contacts on steep terrain
            try:
                p.setPhysicsEngineParameter(numSolverIterations=int(demo_solver_iters),
physicsClientId=pid)
            except Exception:
                pass

            # Higher friction on the environment so it can grip the slope
            for env_id in [mountain_id, floor_id]:
                try:
                    p.changeDynamics(
                        env_id, -1,
                        lateralFriction=float(demo_env_friction),
                        restitution=0.0,
                        physicsClientId=pid
                    )
                except Exception:
                    pass

            # Tune damping and friction on the creature links
            n_joints = p.getNumJoints(cid, physicsClientId=pid)

            # Base link is index -1 in PyBullet
            try:
                p.changeDynamics(
                    cid, -1,
                    lateralFriction=float(demo_link_friction),
                    linearDamping=float(demo_linear_damping),
                    angularDamping=float(demo_angular_damping),
                    restitution=0.0,
                    physicsClientId=pid
                )
```

```
        except Exception:
            pass

        for j in range(n_joints):
            try:
                p.changeDynamics(
                    cid, j,
                    lateralFriction=float(demo_link_friction),
                    linearDamping=float(demo_linear_damping),
                    angularDamping=float(demo_angular_damping),
                    restitution=0.0,
                    physicsClientId=pid
                )
            except Exception:
                pass

    # Metrics collected for logging
    # supported_min_dist: only count best distance when the creature is supported (in contact).
    # Supported means it has contact with the ground or mountain.
    min_dist_to_peak = float("inf")
    supported_min_dist_to_peak = float("inf")
    max_z = float("-inf")

    air_steps = 0
    consecutive_air = 0
    max_consecutive_air = 0

    for step in range(iterations):
        p.stepSimulation(physicsClientId=pid)

        if control_period > 0 and (step % control_period == 0):
            vel_scale = float(demo_vel_scale) if demo_mode else 1.0
            self.update_motors(cid=cid, cr=cr, motor_force=motor_force, vel_scale=vel_scale)


        if realtime:
            time.sleep(sleep_dt)

        pos, orn = p.getBasePositionAndOrientation(cid, physicsClientId=pid)

        supported = len(p.getContactPoints(bodyA=cid, physicsClientId=pid)) > 0
        if supported:
            consecutive_air = 0
        else:
            if step >= settle_steps:
                air_steps += 1
                consecutive_air += 1
                if consecutive_air > max_consecutive_air:
                    max_consecutive_air = consecutive_air

        cr.update_position(pos)
```

```
            if pos[2] > max_z:
                max_z = pos[2]

            dx = pos[0] - peak[0]
            dy = pos[1] - peak[1]
            dz = pos[2] - peak[2]
            dist = (dx*dx + dy*dy + dz*dz) ** 0.5

            if dist < min_dist_to_peak:
                min_dist_to_peak = dist

            if supported and dist < supported_min_dist_to_peak:
                supported_min_dist_to_peak = dist

        # Save metrics onto creature
        cr.peak_point = peak
        cr.min_dist_to_peak = min_dist_to_peak
        cr.max_z = max_z
        cr.supported_min_dist_to_peak = supported_min_dist_to_peak

        effective_steps = max(1, iterations - settle_steps)
        cr.air_ratio = air_steps / float(effective_steps)
        cr.max_consecutive_air = max_consecutive_air

        # Fitness: supported distance with a small airtime penalty
        # Fitness uses distance to peak, but only when supported to reduce flying exploits.
        # Airtime ratio and max airtime streak help flag flying behaviour.
        if supported_min_dist_to_peak == float("inf"):
            base_fitness = 0.0
        else:
            base_fitness = 1.0 / (supported_min_dist_to_peak + 1e-6)

        flight_penalty = 0.25 * (max_consecutive_air / float(effective_steps))
        cr.fitness = max(0.0, base_fitness - flight_penalty)

    def update_motors(self, cid, cr, motor_force=5, vel_scale=1.0):
        for jid in range(p.getNumJoints(cid, physicsClientId=self.physicsClientId)):
            m = cr.get_motors()[jid]
            p.setJointMotorControl2(
                cid, jid,
                controlMode=p.VELOCITY_CONTROL,
                targetVelocity=float(vel_scale) * float(m.get_output()),
                force=float(motor_force),
                physicsClientId=self.physicsClientId
            )

    # Changed: eval_population now respects the passed evaluation length (iterations),
    # which is needed for controlled experiments in the report.
    def eval_population(self, pop, iterations):
        for cr in pop.creatures:
```

```
            self.run_creature(cr, iterations)


class ThreadedSim():
    def __init__(self, pool_size):
        self.sims = [Simulation(i) for i in range(pool_size)]

    @staticmethod
    def static_run_creature(sim, cr, iterations):
        sim.run_creature(cr, iterations)
        return cr

    def eval_population(self, pop, iterations):
        """
        pop is a Population object
        iterations is frames in pybullet to run for at 240fps
        """
        pool_args = []
        start_ind = 0
        pool_size = len(self.sims)
        while start_ind < len(pop.creatures):
            this_pool_args = []
            for i in range(start_ind, start_ind + pool_size):
                if i == len(pop.creatures):# the end
                    break
                # work out the sim ind
                sim_ind = i % len(self.sims)
                this_pool_args.append([
                        self.sims[sim_ind],
                        pop.creatures[i],
                        iterations]
                )
            pool_args.append(this_pool_args)
            start_ind = start_ind + pool_size

        new_creatures = []
        for pool_argset in pool_args:
            with Pool(pool_size) as p:
                # it works on a copy of the creatures, so receive them
                creatures = p.starmap(ThreadedSim.static_run_creature, pool_argset)
                # and now put those creatures back into the main
                # self.creatures array
                new_creatures.extend(creatures)
        pop.creatures = new_creatures
```

genome.py

File: genome.py

Highlighted lines indicate differences relative to the starter codebase.

```
import numpy as np
import copy
import random

class Genome():
    @staticmethod
    def get_random_gene(length):
        gene = np.array([np.random.random() for i in range(length)])
        return gene

    @staticmethod
    def get_random_genome(gene_length, gene_count):
        genome = [Genome.get_random_gene(gene_length) for i in range(gene_count)]
        return genome

    @staticmethod
    def get_gene_spec():
        gene_spec =  {"link-shape":{"scale":1},
            "link-length": {"scale":2},
            "link-radius": {"scale":1},
            "link-recurrence": {"scale":3},
            "link-mass": {"scale":1},
            "joint-type": {"scale":1},
            "joint-parent":{"scale":1},
            "joint-axis-xyz": {"scale":1},
            "joint-origin-rpy-1":{"scale":np.pi * 2},
            "joint-origin-rpy-2":{"scale":np.pi * 2},
            "joint-origin-rpy-3":{"scale":np.pi * 2},
            "joint-origin-xyz-1":{"scale":1},
            "joint-origin-xyz-2":{"scale":1},
            "joint-origin-xyz-3":{"scale":1},
            "control-waveform":{"scale":1},
            "control-amp":{"scale":0.25},
            "control-freq":{"scale":1}
            }
        ind = 0
        for key in gene_spec.keys():
            gene_spec[key]["ind"] = ind
            ind = ind + 1
        return gene_spec

    @staticmethod
    def get_gene_dict(gene, spec):
```

```
        gdict = {}
        for key in spec:
            ind = spec[key]["ind"]
            scale = spec[key]["scale"]
            gdict[key] = gene[ind] * scale
        return gdict

    @staticmethod
    def get_genome_dicts(genome, spec):
        gdicts = []
        for gene in genome:
            gdicts.append(Genome.get_gene_dict(gene, spec))
        return gdicts

    @staticmethod
    def expandLinks(parent_link, uniq_parent_name, flat_links, exp_links):
        children = [l for l in flat_links if l.parent_name == parent_link.name]
        sibling_ind = 1
        for c in children:
            for r in range(int(c.recur)):
                sibling_ind  = sibling_ind +1
                c_copy = copy.copy(c)
                c_copy.parent_name = uniq_parent_name
                uniq_name = c_copy.name + str(len(exp_links))
                #print("exp: ", c.name, " -> ", uniq_name)
                c_copy.name = uniq_name
                c_copy.sibling_ind = sibling_ind
                exp_links.append(c_copy)
                assert c.parent_name != c.name, "Genome::expandLinks: link joined to itself: " + c.name + "
joins " + c.parent_name
                Genome.expandLinks(c, uniq_name, flat_links, exp_links)

    @staticmethod
    def genome_to_links(gdicts):
        links = []
        link_ind = 0
        parent_names = [str(link_ind)]
        for gdict in gdicts:
            link_name = str(link_ind)
            parent_ind = gdict["joint-parent"] * len(parent_names)
            assert parent_ind < len(parent_names), "genome.py: parent ind too high: " + str(parent_ind) +
"got: " + str(parent_names)
            parent_name = parent_names[int(parent_ind)]
            #print("available parents: ", parent_names, "chose", parent_name)
            recur = gdict["link-recurrence"]
            link = URDFLink(name=link_name,
                            parent_name=parent_name,
                            recur=recur+1,
                            link_length=gdict["link-length"],
                            link_radius=gdict["link-radius"],
                            link_mass=gdict["link-mass"],
```

```
                    joint_type=gdict["joint-type"],
                    joint_parent=gdict["joint-parent"],
                    joint_axis_xyz=gdict["joint-axis-xyz"],
                    joint_origin_rpy_1=gdict["joint-origin-rpy-1"],
                    joint_origin_rpy_2=gdict["joint-origin-rpy-2"],
                    joint_origin_rpy_3=gdict["joint-origin-rpy-3"],
                    joint_origin_xyz_1=gdict["joint-origin-xyz-1"],
                    joint_origin_xyz_2=gdict["joint-origin-xyz-2"],
                    joint_origin_xyz_3=gdict["joint-origin-xyz-3"],
                    control_waveform=gdict["control-waveform"],
                    control_amp=gdict["control-amp"],
                    control_freq=gdict["control-freq"],
                    # Added shape links
                    link_shape=gdict["link-shape"])
        links.append(link)
        if link_ind != 0:# don't re-add the first link
            parent_names.append(link_name)
        link_ind = link_ind + 1

    # now just fix the first link so it links to nothing
    links[0].parent_name = "None"
    return links

@staticmethod
def crossover(g1, g2):
    x1 = random.randint(0, len(g1)-1)
    x2 = random.randint(0, len(g2)-1)
    g3 = np.concatenate((g1[x1:], g2[x2:]))
    if len(g3) > len(g1):
        g3 = g3[0:len(g1)]
    return g3

@staticmethod
def point_mutate(genome, rate, amount):
    new_genome = copy.copy(genome)
    for gene in new_genome:
        for i in range(len(gene)):
            if random.random() < rate:
                # Changed: use the 'amount' parameter so mutation strength is controllable in
experiments
                gene[i] += (random.random() * 2 - 1) * amount
            if gene[i] >= 1.0:
                gene[i] = 0.9999
            if gene[i] < 0.0:
                gene[i] = 0.0
    return new_genome

@staticmethod
def shrink_mutate(genome, rate):
    if len(genome) == 1:
        return copy.copy(genome)
```

```
        if random.random() < rate:
            ind = random.randint(0, len(genome)-1)
            new_genome = np.delete(genome, ind, 0)
            return new_genome
        else:
            return copy.copy(genome)

    @staticmethod
    def grow_mutate(genome, rate):
        if random.random() < rate:
            gene = Genome.get_random_gene(len(genome[0]))
            new_genome = copy.copy(genome)
            new_genome = np.append(new_genome, [gene], axis=0)
            return new_genome
        else:
            return copy.copy(genome)


    @staticmethod
    def to_csv(dna, csv_file):
        csv_str = ""
        for gene in dna:
            for val in gene:
                csv_str = csv_str + str(val) + ","
            csv_str = csv_str + '\n'

        with open(csv_file, 'w') as f:
            f.write(csv_str)

    @staticmethod
    def from_csv(filename):
        csv_str = ''
        with open(filename) as f:
            csv_str = f.read()
        dna = []
        lines = csv_str.split('\n')
        for line in lines:
            vals = line.split(',')
            gene = [float(v) for v in vals if v != '']
            if len(gene) > 0:
                dna.append(gene)
        return dna

class URDFLink:
    def __init__(self, name, parent_name, recur,
                 link_shape=0.0,
                 link_length=0.1,
                 link_radius=0.1,
                 link_mass=0.1,
                 joint_type=0.1,
                 joint_parent=0.1,
```

```
            joint_axis_xyz=0.1,
            joint_origin_rpy_1=0.1,
            joint_origin_rpy_2=0.1,
            joint_origin_rpy_3=0.1,
            joint_origin_xyz_1=0.1,
            joint_origin_xyz_2=0.1,
            joint_origin_xyz_3=0.1,
            control_waveform=0.1,
            control_amp=0.1,
            control_freq=0.1):
        self.name = name
        self.parent_name = parent_name
        self.recur = recur
        self.link_length=link_length
        self.link_radius=link_radius
        self.link_mass=link_mass
        self.joint_type=joint_type
        self.joint_parent=joint_parent
        self.joint_axis_xyz=joint_axis_xyz
        self.joint_origin_rpy_1=joint_origin_rpy_1
        self.joint_origin_rpy_2=joint_origin_rpy_2
        self.joint_origin_rpy_3=joint_origin_rpy_3
        self.joint_origin_xyz_1=joint_origin_xyz_1
        self.joint_origin_xyz_2=joint_origin_xyz_2
        self.joint_origin_xyz_3=joint_origin_xyz_3
        self.control_waveform=control_waveform
        self.control_amp=control_amp
        self.control_freq=control_freq
        self.sibling_ind = 1
        # Link shape
        self.link_shape = link_shape

    # Encoding scheme extension:
    # link-shape gene selects between cylinder, box, and sphere geometry.
    # This widens the morphology search space for advanced experiments.
    def to_link_element(self, adom):
        # Added: decode link_shape into different URDF geometries (encoding experiment)
        # link_shape in [0,1): cylinder / box / sphere based on thresholds

        # Conservative clamping to avoid physics instability
        length = max(0.05, float(self.link_length))
        radius = max(0.02, float(self.link_radius))

        # Decide shape
        # Shape thresholds split [0, 1] into three regions for cylinder, box, sphere.
        # This keeps shape selection simple while remaining evolvable.
        if self.link_shape < 0.33:
            shape_type = "cylinder"
        elif self.link_shape < 0.66:
            shape_type = "box"
        else:
```

```
        shape_type = "sphere"

    link_tag = adom.createElement("link")
    link_tag.setAttribute("name", self.name)

    # ---------- Visual ----------
    vis_tag = adom.createElement("visual")
    geom_tag = adom.createElement("geometry")

    if shape_type == "cylinder":
        shape_tag = adom.createElement("cylinder")
        shape_tag.setAttribute("length", str(length))
        shape_tag.setAttribute("radius", str(radius))
        volume = np.pi * (radius ** 2) * length

    elif shape_type == "box":
        shape_tag = adom.createElement("box")
        # Use length for x, radius for y/z thickness
        sx = length
        sy = 2 * radius
        sz = 2 * radius
        shape_tag.setAttribute("size", f"{sx} {sy} {sz}")
        volume = sx * sy * sz

    else:  # sphere
        shape_tag = adom.createElement("sphere")
        shape_tag.setAttribute("radius", str(radius))
        volume = (4.0 / 3.0) * np.pi * (radius ** 3)

    geom_tag.appendChild(shape_tag)
    vis_tag.appendChild(geom_tag)

    # ---------- Collision ----------
    coll_tag = adom.createElement("collision")
    c_geom_tag = adom.createElement("geometry")

    # Reuse the same shape for collision
    c_shape_tag = adom.createElement(shape_type)
    if shape_type == "cylinder":
        c_shape_tag.setAttribute("length", str(length))
        c_shape_tag.setAttribute("radius", str(radius))
    elif shape_type == "box":
        c_shape_tag.setAttribute("size", f"{length} {2*radius} {2*radius}")
    else:
        c_shape_tag.setAttribute("radius", str(radius))

    c_geom_tag.appendChild(c_shape_tag)
    coll_tag.appendChild(c_geom_tag)

    # ---------- Inertial ----------
    # Approximate volume and inertia based on chosen geometry so physics remains plausible.
```

```
        # Clamp values to avoid unstable bodies due to extreme gene combinations.
        inertial_tag = adom.createElement("inertial")
        mass_tag = adom.createElement("mass")

        # Changed: mass now depends on decoded shape volume and the link-mass gene
        base_density = 1.0

        # link_mass comes from gene in [0, 1). Map it to a stable multiplier range.
        # This keeps masses from going to ~0 while still allowing meaningful variation.
        mass_mult = 0.5 + 1.5 * float(self.link_mass)   # range ~[0.5, 2.0)

        density = base_density * mass_mult
        mass = density * volume
        mass_tag.setAttribute("value", str(mass))
        inertia_tag = adom.createElement("inertia")
        # Compute principal inertias for common shapes assuming uniform density.
        # We align cylinder length along x to match how the "length" is used for box size too.
        if shape_type == "cylinder":
            r = radius
            l = length
            ixx = (1.0 / 12.0) * mass * (3 * r * r + l * l)
            iyy = ixx
            izz = 0.5 * mass * r * r

        elif shape_type == "box":
            sx = length
            sy = 2 * radius
            sz = 2 * radius
            ixx = (1.0 / 12.0) * mass * (sy * sy + sz * sz)
            iyy = (1.0 / 12.0) * mass * (sx * sx + sz * sz)
            izz = (1.0 / 12.0) * mass * (sx * sx + sy * sy)

        else:  # sphere
            r = radius
            ixx = 0.4 * mass * r * r
            iyy = ixx
            izz = ixx

        inertia_tag.setAttribute("ixx", str(ixx))
        inertia_tag.setAttribute("iyy", str(iyy))
        inertia_tag.setAttribute("izz", str(izz))
        inertia_tag.setAttribute("ixy", "0")
        inertia_tag.setAttribute("ixz", "0")
        inertia_tag.setAttribute("iyz", "0")  # Fixed: URDF uses iyz, not iyx

        # attach mass and inertia to the inertial tag
        inertial_tag.appendChild(mass_tag)
        inertial_tag.appendChild(inertia_tag)

        link_tag.appendChild(vis_tag)
        link_tag.appendChild(coll_tag)
```

```
        link_tag.appendChild(inertial_tag)
        return link_tag

    def to_joint_element(self, adom):
        #           <joint name="base_to_sub2" type="revolute">
        #     <parent link="base_link"/>
        #     <child link="sub_link2"/>
        #     <axis xyz="1 0 0"/>
        #     <limit effort="10" upper="0" lower="10" velocity="1"/>
        #     <origin rpy="0 0 0" xyz="0 0.5 0"/>
        #   </joint>
        joint_tag = adom.createElement("joint")
        joint_tag.setAttribute("name", self.name + "_to_" + self.parent_name)
        if self.joint_type >= 0.5:
            joint_tag.setAttribute("type", "revolute")
        else:
            joint_tag.setAttribute("type", "revolute")
        parent_tag = adom.createElement("parent")
        parent_tag.setAttribute("link", self.parent_name)
        child_tag = adom.createElement("child")
        child_tag.setAttribute("link", self.name)
        axis_tag = adom.createElement("axis")
        if self.joint_axis_xyz <= 0.33:
            axis_tag.setAttribute("xyz", "1 0 0")
        if self.joint_axis_xyz > 0.33 and self.joint_axis_xyz <= 0.66:
            axis_tag.setAttribute("xyz", "0 1 0")
        if self.joint_axis_xyz > 0.66:
            axis_tag.setAttribute("xyz", "0 0 1")

        limit_tag = adom.createElement("limit")
        # effort upper lower velocity
        limit_tag.setAttribute("effort", "1")
        limit_tag.setAttribute("upper", "3.1415")
        limit_tag.setAttribute("lower", "-3.1415")
        limit_tag.setAttribute("velocity", "1")
        # <origin rpy="0 0 0" xyz="0 0.5 0"/>
        orig_tag = adom.createElement("origin")

        rpy1 = self.joint_origin_rpy_1 * self.sibling_ind
        rpy = str(rpy1) + " " + str(self.joint_origin_rpy_2) + " " + str(self.joint_origin_rpy_3)

        orig_tag.setAttribute("rpy", rpy)
        xyz = str(self.joint_origin_xyz_1) + " " + str(self.joint_origin_xyz_2) + " " +
str(self.joint_origin_xyz_3)
        orig_tag.setAttribute("xyz", xyz)

        joint_tag.appendChild(parent_tag)
        joint_tag.appendChild(child_tag)
        joint_tag.appendChild(axis_tag)
        joint_tag.appendChild(limit_tag)
        joint_tag.appendChild(orig_tag)
```

```
return joint_tag
```

creature.py

File: creature.py

Highlighted lines indicate differences relative to the starter codebase.

```
import genome
from xml.dom.minidom import getDOMImplementation
from enum import Enum
import numpy as np

class MotorType(Enum):
    PULSE = 1
    SINE = 2

class Motor:
    def __init__(self, control_waveform, control_amp, control_freq):
        if control_waveform <= 0.5:
            self.motor_type = MotorType.PULSE
        else:
            self.motor_type = MotorType.SINE
        self.amp = control_amp
        self.freq = control_freq
        self.phase = 0


    def get_output(self):
        self.phase = (self.phase + self.freq) % (np.pi * 2)
        if self.motor_type == MotorType.PULSE:
            if self.phase < np.pi:
                output = 1
            else:
                output = -1

        if self.motor_type == MotorType.SINE:
            output = np.sin(self.phase)

        return output

class Creature:
    def __init__(self, gene_count, control_scheme="per_joint"):
        # control_scheme controls how motor parameters are encoded.
        # per_joint encodes each joint independently, global shares one motor profile across joints.
        self.control_scheme = control_scheme
        self.spec = genome.Genome.get_gene_spec()
        self.dna = genome.Genome.get_random_genome(len(self.spec), gene_count)
        self.flat_links = None
        self.exp_links = None
        self.motors = None
```

```
        self.start_position = None
        self.last_position = None

    def get_flat_links(self):
        if self.flat_links == None:
            gdicts = genome.Genome.get_genome_dicts(self.dna, self.spec)
            self.flat_links = genome.Genome.genome_to_links(gdicts)
        return self.flat_links

    def get_expanded_links(self):
        self.get_flat_links()
        if self.exp_links is not None:
            return self.exp_links

        exp_links = [self.flat_links[0]]
        genome.Genome.expandLinks(self.flat_links[0],
                                  self.flat_links[0].name,
                                  self.flat_links,
                                  exp_links)
        self.exp_links = exp_links
        return self.exp_links

    def to_xml(self):
        self.get_expanded_links()
        domimpl = getDOMImplementation()
        adom = domimpl.createDocument(None, "start", None)
        robot_tag = adom.createElement("robot")
        for link in self.exp_links:
            robot_tag.appendChild(link.to_link_element(adom))
        first = True
        for link in self.exp_links:
            if first:# skip the root node!
                first = False
                continue
            robot_tag.appendChild(link.to_joint_element(adom))
        robot_tag.setAttribute("name", "pepe") #  choose a name!
        return '<?xml version="1.0"?>' + robot_tag.toprettyxml()

    # Motor decoding:
    # - global uses the same waveform, amplitude, and frequency for all joints
    # - per_joint uses each link's own control genes
    def get_motors(self):
        self.get_expanded_links()
        if self.motors is None:
            motors = []

            # Added: global motor control scheme reduces degrees of freedom (encoding experiment)
            if self.control_scheme == "global" and len(self.exp_links) > 1:
                # Use the first non-root link as the reference oscillator for global control
                ref = self.exp_links[1]
                wave = ref.control_waveform
```

```
            amp = ref.control_amp
            freq = ref.control_freq
            for _ in range(1, len(self.exp_links)):
                motors.append(Motor(wave, amp, freq))
        else:
            # Default: per-joint motor parameters
            for i in range(1, len(self.exp_links)):
                l = self.exp_links[i]
                motors.append(Motor(l.control_waveform, l.control_amp, l.control_freq))

        self.motors = motors
    return self.motors

def update_position(self, pos):
    if self.start_position == None:
        self.start_position = pos
    else:
        self.last_position = pos

def get_distance_travelled(self):
    if self.start_position is None or self.last_position is None:
        return 0
    p1 = np.asarray(self.start_position)
    p2 = np.asarray(self.last_position)
    dist = np.linalg.norm(p1-p2)
    return dist

def update_dna(self, dna):
    self.dna = dna
    self.flat_links = None
    self.exp_links = None
    self.motors = None
    self.start_position = None
    self.last_position = None
```

population.py

File: population.py

Highlighted lines indicate differences relative to the starter codebase.

```
import creature
import numpy as np

# Added: roulette-wheel selection helpers using cr.fitness.
# Used by run_ga_mountain.py to pick parents each generation.
class Population:
    def __init__(self, pop_size, gene_count):
        self.creatures = [
            creature.Creature(gene_count=gene_count)
            for _ in range(pop_size)
        ]

    # Added: convert fitness values to a cumulative map for roulette selection.
    # Also clamps negative or NaN values to avoid crashes.
    @staticmethod
    def get_fitness_map(fits):
        """
        Convert a list of fitness values into a cumulative fitness map
        for roulette-wheel selection.
        """
        fits = np.array(fits, dtype=float)

        # Guard against negative or NaN fitness
        fits = np.nan_to_num(fits, nan=0.0)
        fits[fits < 0] = 0.0

        return np.cumsum(fits)

    # Updated: roulette-wheel parent selection with a fallback.
    # If total fitness is zero (early generations), select uniformly at random.
    @staticmethod
    def select_parent(fitmap):
        """
        Roulette-wheel selection using cumulative fitmap.
        If total fitness is zero, fall back to uniform random selection.
        """
        total = float(fitmap[-1]) if len(fitmap) > 0 else 0.0
        if total <= 0.0:
            return np.random.randint(0, len(fitmap))

        r = np.random.rand() * total
        for i in range(len(fitmap)):
            if r <= fitmap[i]:
```

```
            return i
        return len(fitmap) - 1
```

run_ga_mountain.py

File: run_ga_mountain.py

Highlighted lines indicate differences relative to the starter codebase.

```
"""
run_ga_mountain.py

Runs GA experiments in the mountain environment and writes evidence for the report.
Outputs per run:
- runs/<run_name>/params.json for reproducibility
- runs/<run_name>/log.csv for plots and tables
- runs/<run_name>/elite/*.csv for demo playback

This script is used for both baseline GA experiments and encoding scheme experiments.
"""

import os
import csv
import time
import numpy as np

# Added for reproducible experiment runs and report evidence
import json

# Added to allow parameter sweeps from the command line without editing code
import argparse

import population
import simulation
import genome
import creature

# Make a unique folder per run so results are not overwritten and can be referenced in the report.
def make_run_dir(prefix="run"):
    stamp = time.strftime("%Y%m%d_%H%M%S")
    run_dir = os.path.join("runs", f"{prefix}_{stamp}")
    elite_dir = os.path.join(run_dir, "elite")
    os.makedirs(elite_dir, exist_ok=True)
    return run_dir, elite_dir


def evaluate_population(sim, pop, iterations):
    # Note: non-threaded eval is more reliable on Windows
    # This avoids multiprocessing issues across different PCs.

    for cr in pop.creatures:
        sim.run_creature(cr, iterations)
```

```
def enforce_encoding(dna, template_dna, spec, encoding, shape_mode):
    # Added: enforce encoding constraints so conditions stay comparable

    # Indices of motor control genes
    control_keys = ["control-waveform", "control-amp", "control-freq"]
    control_inds = [spec[k]["ind"] for k in control_keys]

    # Morphology-related keys to freeze for "control_only"
    morph_keys = [k for k in spec.keys() if k not in control_keys]
    morph_inds = [spec[k]["ind"] for k in morph_keys]

    new_dna = []
    for gi, gene in enumerate(dna):
        g = np.array(gene, dtype=float)

        # shape_mode="cylinder_only": force link geometry to cylinders so shape does not vary
        if shape_mode == "cylinder_only":
            shape_ind = spec["link-shape"]["ind"]
            g[shape_ind] = 0.10  # always selects cylinder in your thresholding

        # encoding="control_only": freeze morphology genes using a fixed template, evolve only motor
genes
        if encoding == "control_only":
             # Freeze morphology using template, keep only control genes from the mutated genome
            t = np.array(template_dna[gi], dtype=float) if gi < len(template_dna) else
np.array(template_dna[-1], dtype=float)
            t[control_inds] = g[control_inds]
            g = t

        # encoding="legs_only": freeze the root/body gene, allow child links to evolve
        elif encoding == "legs_only":
            # Freeze the first gene (body/root) completely, allow others to evolve
            if gi == 0 and len(template_dna) > 0:
                g = np.array(template_dna[0], dtype=float)

        new_dna.append(g)

    return new_dna

def main():
    # ------------------------
    # Experiment parameters
    # ------------------------
    # Added: CLI args for controlled experiments
    # This allows changing population size, mutation rates, seeds, and iteration length per run.

    parser = argparse.ArgumentParser()
    parser.add_argument("--seed", type=int, default=1)
    parser.add_argument("--pop_size", type=int, default=25)
    parser.add_argument("--gene_count", type=int, default=3)
```

```
    parser.add_argument("--generations", type=int, default=100)
    parser.add_argument("--iterations", type=int, default=2400)

    parser.add_argument("--point_rate", type=float, default=0.10)
    parser.add_argument("--point_amount", type=float, default=0.25)
    parser.add_argument("--shrink_rate", type=float, default=0.25)
    parser.add_argument("--grow_rate", type=float, default=0.10)

    # Added: encoding experiment toggles (shape, motor scheme, and partial evolvability)
    parser.add_argument("--shape_mode", type=str, default="mixed", choices=["mixed",
"cylinder_only"])
    parser.add_argument("--control_scheme", type=str, default="per_joint", choices=["per_joint",
"global"])
    parser.add_argument("--encoding", type=str, default="full", choices=["full", "legs_only",
"control_only"])

    # Read experiment settings from the command line
    args = parser.parse_args()

    seed = args.seed
    pop_size = args.pop_size
    gene_count = args.gene_count
    generations = args.generations
    iterations = args.iterations
    point_rate = args.point_rate
    point_amount = args.point_amount
    shrink_rate = args.shrink_rate
    grow_rate = args.grow_rate
    shape_mode = args.shape_mode
    control_scheme = args.control_scheme
    encoding = args.encoding

    np.random.seed(seed)  # Added: ensure reproducibility per seed

    # Added: unique run directory so logs and elites do not overwrite each other
    # Added: include encoding settings in the run folder name for clearer comparisons
    run_dir, elite_dir = make_run_dir(

prefix=f"pop{pop_size}_genes{gene_count}_seed{seed}_{shape_mode}_{control_scheme}_{encoding}
"
    )
    # Added: save full run configuration for reproducibility and for representation in the report
    params = {
        "seed": seed,
        "pop_size": pop_size,
        "gene_count": gene_count,
        "generations": generations,
        "iterations": iterations,
        "point_rate": point_rate,
        "point_amount": point_amount,
        "shrink_rate": shrink_rate,
```

```
        "grow_rate": grow_rate,
        "shape_mode": shape_mode,
        "control_scheme": control_scheme,
        "encoding": encoding,
    }

    # Save config so the run can be reproduced later.
    with open(os.path.join(run_dir, "params.json"), "w") as f:
        json.dump(params, f, indent=2)
    log_path = os.path.join(run_dir, "log.csv")

    pop = population.Population(pop_size=pop_size, gene_count=gene_count)
    # Added: apply encoding constraints to the initial population so generation 0 matches the
experimental condition

    # Added: enforce encoding constraints so experiments are comparable.
    # This ensures the genome is interpreted under the chosen encoding mode, rather than drifting
between modes.
    for cr in pop.creatures:
        fixed_dna = enforce_encoding(cr.dna, cr.dna, cr.spec, encoding, shape_mode)
        cr.update_dna(fixed_dna)
        # Ensure the control scheme is applied consistently across the population
        cr.control_scheme = control_scheme
    sim = simulation.Simulation()

    # CSV logging header
    # Added: log one row per generation for plots and tables in the report
    with open(log_path, "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow([
            "generation",
            "best_fitness",
            "mean_fitness",
            "median_fitness",
            "best_supported_min_dist",
            "best_air_ratio",
            "mean_links",
            "max_links"
        ])

    for gen in range(generations):
        evaluate_population(sim, pop, iterations)

        fits = [getattr(cr, "fitness", 0.0) for cr in pop.creatures]
        links = [len(cr.get_expanded_links()) for cr in pop.creatures]

        best_idx = int(np.argmax(fits))
        best_cr = pop.creatures[best_idx]

        best_supported = getattr(best_cr, "supported_min_dist_to_peak", float("inf"))
        best_air = getattr(best_cr, "air_ratio", 1.0)
```

```
    # Append the generation log row
    # One row per generation for later plots and summary stats
    with open(log_path, "a", newline="") as f:
        writer = csv.writer(f)
        writer.writerow([
            gen,
            float(np.max(fits)),
            float(np.mean(fits)),
            float(np.median(fits)),
            float(best_supported),
            float(best_air),
            float(np.mean(links)),
            float(np.max(links))
        ])

    print(gen,
          "best fitness:", np.round(np.max(fits), 3),
          "mean:", np.round(np.mean(fits), 3),
          "best supported dist:", np.round(best_supported, 3),
          "best air ratio:", np.round(best_air, 3))

    # Save elite DNA each generation
    elite_path = os.path.join(elite_dir, f"elite_gen_{gen:04d}.csv")
    genome.Genome.to_csv(best_cr.dna, elite_path)

    # Roulette-wheel selection map
    fit_map = population.Population.get_fitness_map(fits)

    # Create the next generation by crossover and mutation
    new_creatures = []
    for _ in range(len(pop.creatures)):
        p1 = pop.creatures[population.Population.select_parent(fit_map)]
        p2 = pop.creatures[population.Population.select_parent(fit_map)]

        dna = genome.Genome.crossover(p1.dna, p2.dna)
        dna = genome.Genome.point_mutate(dna, rate=point_rate, amount=point_amount)
        dna = genome.Genome.shrink_mutate(dna, rate=shrink_rate)
        dna = genome.Genome.grow_mutate(dna, rate=grow_rate)
        # Added: apply encoding constraints (shape/motor evolvability experiments)
        dna = enforce_encoding(dna, p1.dna, pop.creatures[0].spec, encoding, shape_mode)

        child = creature.Creature(gene_count=gene_count, control_scheme=control_scheme)
        child.update_dna(dna)
        new_creatures.append(child)

    # Elitism: keep the best creature so the run does not lose progress
    elite = creature.Creature(gene_count=gene_count, control_scheme=control_scheme)
    elite.update_dna(best_cr.dna)
    new_creatures[0] = elite
```

```
        pop.creatures = new_creatures

    print("Run complete.")
    print("Log:", log_path)
    print("Elites:", elite_dir)


if __name__ == "__main__":
    main()
```

plot_runs.py

File: plot_runs.py

Highlighted lines indicate differences relative to the starter codebase.

```
import os
import csv
import argparse

import matplotlib
matplotlib.use("Agg")

import matplotlib.pyplot as plt


def read_log_csv(log_path):
    gens = []
    best_fit = []
    mean_fit = []
    best_dist = []

    with open(log_path, "r", newline="") as f:
        reader = csv.DictReader(f)
        for row in reader:
            gens.append(int(row["generation"]))
            best_fit.append(float(row["best_fitness"]))
            mean_fit.append(float(row["mean_fitness"]))
            best_dist.append(float(row["best_supported_min_dist"]))
    return gens, best_fit, mean_fit, best_dist


def ensure_dir(path):
    os.makedirs(path, exist_ok=True)


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument(
        "run_dirs",
        nargs="+",
        help="One or more run directories that contain log.csv"
    )
    parser.add_argument(
        "--out",
        default="plots",
        help="Output folder for PNG plots (default: plots)"
    )
    args = parser.parse_args()
```

```
    ensure_dir(args.out)

    # Plot 1: Best fitness vs generation (all runs on one chart)
    plt.figure()
    for run_dir in args.run_dirs:
        log_path = os.path.join(run_dir, "log.csv")
        if not os.path.exists(log_path):
            raise FileNotFoundError(f"Missing log.csv in {run_dir}")
        gens, best_fit, mean_fit, best_dist = read_log_csv(log_path)
        label = os.path.basename(run_dir.rstrip("\\/"))
        plt.plot(gens, best_fit, label=label)
    plt.xlabel("Generation")
    plt.ylabel("Best fitness")
    plt.legend()
    plt.tight_layout()
    plt.savefig(os.path.join(args.out, "best_fitness_vs_generation.png"), dpi=200)
    plt.close()

    # Plot 2: Best supported distance vs generation (lower is better)
    plt.figure()
    for run_dir in args.run_dirs:
        log_path = os.path.join(run_dir, "log.csv")
        gens, best_fit, mean_fit, best_dist = read_log_csv(log_path)
        label = os.path.basename(run_dir.rstrip("\\/"))
        plt.plot(gens, best_dist, label=label)
    plt.xlabel("Generation")
    plt.ylabel("Best supported distance to peak")
    plt.legend()
    plt.tight_layout()
    plt.savefig(os.path.join(args.out, "best_supported_distance_vs_generation.png"), dpi=200)
    plt.close()

    # Plot 3: Mean fitness vs generation
    plt.figure()
    for run_dir in args.run_dirs:
        log_path = os.path.join(run_dir, "log.csv")
        gens, best_fit, mean_fit, best_dist = read_log_csv(log_path)
        label = os.path.basename(run_dir.rstrip("\\/"))
        plt.plot(gens, mean_fit, label=label)
    plt.xlabel("Generation")
    plt.ylabel("Mean fitness")
    plt.legend()
    plt.tight_layout()
    plt.savefig(os.path.join(args.out, "mean_fitness_vs_generation.png"), dpi=200)
    plt.close()

    print("Saved plots to:", os.path.abspath(args.out))


if __name__ == "__main__":
```

main()

plot_air_ratio.py

File: plot_air_ratio.py

Highlighted lines indicate differences relative to the starter codebase.

```
import csv
import os
import argparse

import matplotlib
matplotlib.use("Agg")

import matplotlib.pyplot as plt

def read_log(run_dir):
    path = os.path.join(run_dir, "log.csv")
    gens, air = [], []
    with open(path, newline="") as f:
        for row in csv.DictReader(f):
            gens.append(int(row["generation"]))
            air.append(float(row["best_air_ratio"]))
    return gens, air

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("run_dirs", nargs="+")
    args = parser.parse_args()

    for run_dir in args.run_dirs:
        gens, air = read_log(run_dir)
        plt.plot(gens, air, label=os.path.basename(run_dir.rstrip("\\/")))

    plt.xlabel("Generation")
    plt.ylabel("Best air ratio (lower is better)")
    plt.title("Best air ratio vs generation")
    plt.legend(fontsize=7)
    os.makedirs("plots", exist_ok=True)
    plt.savefig(os.path.join("plots", "best_air_ratio_vs_generation.png"), dpi=200, bbox_inches="tight")

if __name__ == "__main__":
    main()
```

summarise_runs.py

File: summarise_runs.py

Highlighted lines indicate differences relative to the starter codebase.

```
import os
import csv
import json
import argparse


def read_last_row(log_path):
    with open(log_path, "r", newline="") as f:
        rows = list(csv.DictReader(f))
    if not rows:
        raise ValueError(f"No rows in {log_path}")
    return rows[-1]


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("run_dirs", nargs="+")
    args = parser.parse_args()

    summary_rows = []
    for run_dir in args.run_dirs:
        log_path = os.path.join(run_dir, "log.csv")
        params_path = os.path.join(run_dir, "params.json")

        last = read_last_row(log_path)

        params = {}
        if os.path.exists(params_path):
            with open(params_path, "r") as f:
                params = json.load(f)

        summary_rows.append({
            "run": os.path.basename(run_dir.rstrip("\\/")),
            "seed": params.get("seed", ""),
            "pop_size": params.get("pop_size", ""),
            "gene_count": params.get("gene_count", ""),
            "iterations": params.get("iterations", ""),
            "generations": params.get("generations", ""),
            "final_best_fitness": last["best_fitness"],
            "final_best_supported_dist": last["best_supported_min_dist"],
            "final_best_air_ratio": last["best_air_ratio"],
            "final_mean_fitness": last["mean_fitness"],
        })
```

```
    # Print as CSV to the terminal for easy copy into report or spreadsheet
    header = list(summary_rows[0].keys())
    print(",".join(header))
    for r in summary_rows:
        print(",".join(str(r[h]) for h in header))


if __name__ == "__main__":
    main()
```

summarise_grouped.py

File: summarise_grouped.py

Highlighted lines indicate differences relative to the starter codebase.

```
import os, csv, json, argparse, statistics

def read_last_row(log_path):
    with open(log_path, "r", newline="") as f:
        rows = list(csv.DictReader(f))
    return rows[-1]

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("run_dirs", nargs="+")
    args = parser.parse_args()

    rows = []
    for run_dir in args.run_dirs:
        last = read_last_row(os.path.join(run_dir, "log.csv"))
        with open(os.path.join(run_dir, "params.json"), "r") as f:
            params = json.load(f)

        rows.append({
            "pop_size": int(params["pop_size"]),
            "seed": int(params["seed"]),
            "final_best_supported_dist": float(last["best_supported_min_dist"]),
            "final_best_air_ratio": float(last["best_air_ratio"]),
            "final_best_fitness": float(last["best_fitness"]),
        })

    # group by pop_size
    by_pop = {}
    for r in rows:
        by_pop.setdefault(r["pop_size"], []).append(r)


print("pop_size,n,mean_final_best_supported_dist,std_final_best_supported_dist,mean_final_best_air
_ratio,std_final_best_air_ratio")
    for pop in sorted(by_pop.keys()):
        grp = by_pop[pop]
        dists = [g["final_best_supported_dist"] for g in grp]
        airs = [g["final_best_air_ratio"] for g in grp]
        print(",".join(map(str, [
            pop,
            len(grp),
            statistics.mean(dists),
            statistics.pstdev(dists) if len(dists) > 1 else 0.0,
```

```
            statistics.mean(airs),
            statistics.pstdev(airs) if len(airs) > 1 else 0.0
        ]))

if __name__ == "__main__":
    main()
```

summarise_grouped_conditions.py

File: summarise_grouped_conditions.py

Highlighted lines indicate differences relative to the starter codebase.

```
import os, csv, json, argparse, statistics

def read_last_row(log_path):
    with open(log_path, "r", newline="") as f:
        rows = list(csv.DictReader(f))
    if not rows:
        raise ValueError(f"No rows in {log_path}")
    return rows[-1]

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("run_dirs", nargs="+")
    args = parser.parse_args()

    rows = []
    for run_dir in args.run_dirs:
        log_path = os.path.join(run_dir, "log.csv")
        params_path = os.path.join(run_dir, "params.json")

        last = read_last_row(log_path)
        with open(params_path, "r") as f:
            params = json.load(f)

        rows.append({
            "run": os.path.basename(run_dir.rstrip("\\/")),
            "seed": int(params.get("seed", -1)),
            "pop_size": int(params.get("pop_size", -1)),
            "shape_mode": params.get("shape_mode", ""),
            "control_scheme": params.get("control_scheme", ""),
            "encoding": params.get("encoding", ""),
            "final_best_supported_dist": float(last["best_supported_min_dist"]),
            "final_best_air_ratio": float(last["best_air_ratio"]),
        })

    # group by condition (shape_mode, control_scheme, encoding)
    by_cond = {}
    for r in rows:
        key = (r["shape_mode"], r["control_scheme"], r["encoding"])
        by_cond.setdefault(key, []).append(r)


print("shape_mode,control_scheme,encoding,n,mean_final_best_supported_dist,std_final_best_suppo
rted_dist,mean_final_best_air_ratio,std_final_best_air_ratio")
```

```
    for (shape_mode, control_scheme, encoding) in sorted(by_cond.keys()):
        grp = by_cond[(shape_mode, control_scheme, encoding)]
        dists = [g["final_best_supported_dist"] for g in grp]
        airs = [g["final_best_air_ratio"] for g in grp]
        print(",".join(map(str, [
            shape_mode,
            control_scheme,
            encoding,
            len(grp),
            statistics.mean(dists),
            statistics.pstdev(dists) if len(dists) > 1 else 0.0,
            statistics.mean(airs),
            statistics.pstdev(airs) if len(airs) > 1 else 0.0
        ])))

if __name__ == "__main__":
    main()
```

demo_gui.py

File: demo_gui.py

Highlighted lines indicate differences relative to the starter codebase.

```
import argparse
import os

import creature
import genome
from simulation import Simulation

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--elite_csv", type=str, required=True, help="Path to elite_gen_XXXX.csv")
    parser.add_argument("--gene_count", type=int, default=3)
    parser.add_argument("--control_scheme", type=str, default="per_joint", choices=["per_joint",
"global"])

    # Demo tuning
    parser.add_argument("--iterations", type=int, default=2400)
    parser.add_argument("--start_x", type=float, default=8.0)
    parser.add_argument("--start_y", type=float, default=0.0)
    parser.add_argument("--start_z", type=float, default=2.0)
    parser.add_argument("--motor_force", type=float, default=25.0)
    parser.add_argument("--control_period", type=int, default=4)
    parser.add_argument("--realtime", action="store_true")
    parser.add_argument("--demo_mode", action="store_true", help="Enable demo-only physics
assists")
    parser.add_argument("--demo_link_friction", type=float, default=2.0)
    parser.add_argument("--demo_env_friction", type=float, default=2.0)
    parser.add_argument("--demo_solver_iters", type=int, default=150)
    parser.add_argument("--demo_vel_scale", type=float, default=8.0)

    args = parser.parse_args()

    dna = genome.Genome.from_csv(args.elite_csv)

    cr = creature.Creature(gene_count=args.gene_count, control_scheme=args.control_scheme)
    # Note: use update_dna so links and motors rebuild correctly
    cr.update_dna(dna)

    sim = Simulation(connection_mode="GUI")
    sim.run_creature(
        cr,
        iterations=args.iterations,
        start_pos=(args.start_x, args.start_y, args.start_z),
        control_period=args.control_period,
```

```
        motor_force=args.motor_force,
        realtime=args.realtime,
        demo_mode=args.demo_mode,
        demo_link_friction=args.demo_link_friction,
        demo_env_friction=args.demo_env_friction,
        demo_solver_iters=args.demo_solver_iters,
        demo_vel_scale=args.demo_vel_scale,
    )


    print("Demo finished.")
    print("supported_min_dist:", getattr(cr, "supported_min_dist_to_peak", None))
    print("air_ratio:", getattr(cr, "air_ratio", None))
    print("fitness:", getattr(cr, "fitness", None))

if __name__ == "__main__":
    main()
```

demo_mountain_gui.py

File: demo_mountain_gui.py

Highlighted lines indicate differences relative to the starter codebase.

```
import os
import argparse

import genome
import creature
from simulation import Simulation


def pick_elite_csv(run_dir: str, gen: int | None) -> str:
    elite_dir = os.path.join(run_dir, "elite")
    assert os.path.isdir(elite_dir), f"Missing elite folder: {elite_dir}"

    if gen is not None:
        path = os.path.join(elite_dir, f"elite_gen_{gen:04d}.csv")
        assert os.path.exists(path), f"Elite file not found: {path}"
        return path

    # default: last generation elite file
    files = [f for f in os.listdir(elite_dir) if f.startswith("elite_gen_") and f.endswith(".csv")]
    assert files, f"No elite_gen_*.csv files found in: {elite_dir}"
    files.sort()
    return os.path.join(elite_dir, files[-1])


def main():
    parser = argparse.ArgumentParser(description="GUI demo replay of an elite creature on the
mountain.")
    parser.add_argument("--run_dir", required=True, help="Path to a run folder that contains
elite/elite_gen_*.csv")
    parser.add_argument("--gen", type=int, default=None, help="Generation to replay. Default is last
elite file.")
    parser.add_argument("--iterations", type=int, default=2400, help="Simulation steps to run.")
    parser.add_argument("--speed", type=float, default=1.0, help="Playback speed. 1.0 is realtime. 2.0
is 2x faster.")
    parser.add_argument("--no_follow", action="store_true", help="Disable camera follow.")
    args = parser.parse_args()

    elite_csv = pick_elite_csv(args.run_dir, args.gen)
    dna = genome.Genome.from_csv(elite_csv)

    # control_scheme matters for how motors are interpreted
    params_path = os.path.join(args.run_dir, "params.json")
    control_scheme = "per_joint"
```

```
    if os.path.exists(params_path):
        import json
        with open(params_path, "r") as f:
            params = json.load(f)
        control_scheme = params.get("control_scheme", control_scheme)

    cr = creature.Creature(gene_count=len(dna), control_scheme=control_scheme)
    cr.dna = dna
    cr.flat_links = None
    cr.exp_links = None
    cr.motors = None

    sim = Simulation(gui=True)
    sim.run_creature(
        cr,
        iterations=args.iterations,
        realtime=True,
        speed=args.speed,
        camera_follow=(not args.no_follow),
    )

    # Print final demo metrics for your video narration
    print("DEMO RESULT")
    print("supported_min_dist_to_peak:", getattr(cr, "supported_min_dist_to_peak", None))
    print("air_ratio:", getattr(cr, "air_ratio", None))
    print("max_consecutive_air:", getattr(cr, "max_consecutive_air", None))
    print("fitness:", getattr(cr, "fitness", None))


if __name__ == "__main__":
    main()
```

cw-envt.py

File: cw-envt.py

Highlighted lines indicate differences relative to the starter codebase.

```
import pybullet as p
import pybullet_data
import time
import numpy as np
import random
import creature

p.connect(p.GUI)
p.setAdditionalSearchPath(pybullet_data.getDataPath())

import random
import pybullet as p
import math

import pybullet as p



def make_mountain(num_rocks=100, max_size=0.25, arena_size=10, mountain_height=5):
    def gaussian(x, y, sigma=arena_size/4):
        """Return the height of the mountain at position (x, y) using a Gaussian function."""
        return mountain_height * math.exp(-((x**2 + y**2) / (2 * sigma**2)))

    for _ in range(num_rocks):
        x = random.uniform(-1 * arena_size/2, arena_size/2)
        y = random.uniform(-1 * arena_size/2, arena_size/2)
        z = gaussian(x, y)  # Height determined by the Gaussian function

        # Adjust the size of the rocks based on height. Higher rocks (closer to the peak) will be smaller.
        size_factor = 1 - (z / mountain_height)
        size = random.uniform(0.1, max_size) * size_factor

        orientation = p.getQuaternionFromEuler([random.uniform(0, 3.14), random.uniform(0, 3.14),
random.uniform(0, 3.14)])
        rock_shape = p.createCollisionShape(p.GEOM_BOX, halfExtents=[size, size, size])
        rock_visual = p.createVisualShape(p.GEOM_BOX, halfExtents=[size, size, size], rgbaColor=[0.5,
0.5, 0.5, 1])
        rock_body = p.createMultiBody(baseMass=0, baseCollisionShapeIndex=rock_shape,
baseVisualShapeIndex=rock_visual, basePosition=[x, y, z], baseOrientation=orientation)



def make_rocks(num_rocks=100, max_size=0.25, arena_size=10):
```

```
    for _ in range(num_rocks):
        x = random.uniform(-1 * arena_size/2, arena_size/2)
        y = random.uniform(-1 * arena_size/2, arena_size/2)
        z = 0.5  # Adjust based on your needs
        size = random.uniform(0.1,max_size)
        orientation = p.getQuaternionFromEuler([random.uniform(0, 3.14), random.uniform(0, 3.14),
random.uniform(0, 3.14)])
        rock_shape = p.createCollisionShape(p.GEOM_BOX, halfExtents=[size, size, size])
        rock_visual = p.createVisualShape(p.GEOM_BOX, halfExtents=[size, size, size], rgbaColor=[0.5,
0.5, 0.5, 1])
        rock_body = p.createMultiBody(baseMass=0, baseCollisionShapeIndex=rock_shape,
baseVisualShapeIndex=rock_visual, basePosition=[x, y, z], baseOrientation=orientation)


def make_arena(arena_size=10, wall_height=1):
    wall_thickness = 0.5
    floor_collision_shape = p.createCollisionShape(shapeType=p.GEOM_BOX,
halfExtents=[arena_size/2, arena_size/2, wall_thickness])
    floor_visual_shape = p.createVisualShape(shapeType=p.GEOM_BOX, halfExtents=[arena_size/2,
arena_size/2, wall_thickness], rgbaColor=[1, 1, 0, 1])
    floor_body = p.createMultiBody(baseMass=0, baseCollisionShapeIndex=floor_collision_shape,
baseVisualShapeIndex=floor_visual_shape, basePosition=[0, 0, -wall_thickness])

    wall_collision_shape = p.createCollisionShape(shapeType=p.GEOM_BOX,
halfExtents=[arena_size/2, wall_thickness/2, wall_height/2])
    wall_visual_shape = p.createVisualShape(shapeType=p.GEOM_BOX, halfExtents=[arena_size/2,
wall_thickness/2, wall_height/2], rgbaColor=[0.7, 0.7, 0.7, 1])  # Gray walls

    # Create four walls
    p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_collision_shape,
baseVisualShapeIndex=wall_visual_shape, basePosition=[0, arena_size/2, wall_height/2])
    p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_collision_shape,
baseVisualShapeIndex=wall_visual_shape, basePosition=[0, -arena_size/2, wall_height/2])

    wall_collision_shape = p.createCollisionShape(shapeType=p.GEOM_BOX,
halfExtents=[wall_thickness/2, arena_size/2, wall_height/2])
    wall_visual_shape = p.createVisualShape(shapeType=p.GEOM_BOX,
halfExtents=[wall_thickness/2, arena_size/2, wall_height/2], rgbaColor=[0.7, 0.7, 0.7, 1])  # Gray walls

    p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_collision_shape,
baseVisualShapeIndex=wall_visual_shape, basePosition=[arena_size/2, 0, wall_height/2])
    p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_collision_shape,
baseVisualShapeIndex=wall_visual_shape, basePosition=[-arena_size/2, 0, wall_height/2])


p.setGravity(0, 0, -10)

arena_size = 20
make_arena(arena_size=arena_size)

#make_rocks(arena_size=arena_size)
```

```
mountain_position = (0, 0, -1)  # Adjust as needed
mountain_orientation = p.getQuaternionFromEuler((0, 0, 0))
p.setAdditionalSearchPath('shapes/')
# mountain = p.loadURDF("mountain.urdf", mountain_position, mountain_orientation,
useFixedBase=1)
# mountain = p.loadURDF("mountain_with_cubes.urdf", mountain_position, mountain_orientation,
useFixedBase=1)

mountain = p.loadURDF("gaussian_pyramid.urdf", mountain_position, mountain_orientation,
useFixedBase=1)

# generate a random creature
cr = creature.Creature(gene_count=3)
# save it to XML
with open('test.urdf', 'w') as f:
    f.write(cr.to_xml())
# load it into the sim
rob1 = p.loadURDF('test.urdf', (0, 0, 10))


p.setRealTimeSimulation(1)
```

prepare_shapes.py

File: prepare_shapes.py

Highlighted lines indicate differences relative to the starter codebase.

```
import random
import math
from noise import pnoise2

# def write_to_obj(filename, vertices, indices):
#     with open(filename, 'w') as f:
#         # Write vertices to the file
#         for vertex in vertices:
#             f.write(f"v {vertex[0]} {vertex[1]} {vertex[2]}\n")

#         # Write faces to the file
#         for face in indices:
#             # OBJ indices start from 1, not 0
#             f.write(f"f {face[0]+1} {face[1]+1} {face[2]+1}\n")

def write_to_obj(filename, vertices, indices):
    with open(filename, 'w') as f:
        # Write vertices to the file
        for vertex in vertices:
            f.write(f"v {vertex[0]} {vertex[1]} {vertex[2]}\n")

        # Write faces to the file
        for face in indices:
            f.write(f"f {face[0]+1} {face[1]+1} {face[2]+1}\n")

def make_pyramid(filename):
    # Define the tetrahedron vertices and indices
    vertices = [
        [-5, -5, 0],  # Base vertex 1
        [5, -5, 0],   # Base vertex 2
        [0, 5, 0],    # Base vertex 3
        [0, 0, 5]     # Apex vertex
    ]
    indices = [
        [0, 1, 2],  # Base triangle
        [0, 1, 3],  # Side triangle 1
        [1, 2, 3],  # Side triangle 2
        [2, 0, 3]   # Side triangle 3
    ]
    # Write the tetrahedron to an OBJ file
    write_to_obj(filename, vertices, indices)
```

```
def make_rocky_moutain(filename):

    # Define the pyramid vertices and indices
    vertices = [
        [-5, -5, 0],  # Base vertex 1
        [5, -5, 0],   # Base vertex 2
        [5, 5, 0],    # Base vertex 3
        [-5, 5, 0],   # Base vertex 4
        [0, 0, 5]     # Apex vertex
    ]
    indices = [
        [0, 1, 4],  # Side triangle 1
        [1, 2, 4],  # Side triangle 2
        [2, 3, 4],  # Side triangle 3
        [3, 0, 4],  # Side triangle 4
        [0, 1, 2],  # Base triangle 1
        [2, 3, 0]   # Base triangle 2
    ]

    # Add random cubes embedded in the pyramid
    num_cubes = 5
    for _ in range(num_cubes):
        size = random.uniform(0.5, 2)
        x = random.uniform(-4, 4)
        y = random.uniform(-4, 4)
        z = random.uniform(0.5, 4.5)

        # Define cube vertices
        cube_vertices = [
            [x-size/2, y-size/2, z-size/2],
            [x+size/2, y-size/2, z-size/2],
            [x+size/2, y+size/2, z-size/2],
            [x-size/2, y+size/2, z-size/2],
            [x-size/2, y-size/2, z+size/2],
            [x+size/2, y-size/2, z+size/2],
            [x+size/2, y+size/2, z+size/2],
            [x-size/2, y+size/2, z+size/2]
        ]

        # Define cube faces
        cube_indices = [
            [0, 1, 2], [2, 3, 0],  # Bottom
            [4, 5, 6], [6, 7, 4],  # Top
            [0, 1, 5], [5, 4, 0],  # Front
            [1, 2, 6], [6, 5, 1],  # Right
            [2, 3, 7], [7, 6, 2],  # Back
            [3, 0, 4], [4, 7, 3]   # Left
        ]

        # Offset cube indices by the current number of vertices
        offset = len(vertices)
```

```
        cube_indices = [[i+offset for i in face] for face in cube_indices]

        # Add cube vertices and indices to the main lists
        vertices.extend(cube_vertices)
        indices.extend(cube_indices)

    # Write the combined vertices and indices to an OBJ file
    write_to_obj(filename, vertices, indices)



def gaussian(x, y, sigma):
    """Return the height of the shape at position (x, y) using a Gaussian function."""
    return math.exp(-((x**2 + y**2) / (2 * sigma**2)))

def generate_gaussian_pyramid(filename, size=10, resolution=0.5, sigma=3):
    vertices = []
    faces = []

    # Generate vertices using the Gaussian function
    for i in range(int(size / resolution)):
        for j in range(int(size / resolution)):
            x = -size/2 + i * resolution
            y = -size/2 + j * resolution
            z = gaussian(x, y, sigma)
            vertices.append([x, y, z])

    # Generate faces by connecting vertices
    for i in range(int(size / resolution) - 1):
        for j in range(int(size / resolution) - 1):
            # Calculate indices for the current square
            bottom_left = i * int(size / resolution) + j
            bottom_right = i * int(size / resolution) + j + 1
            top_left = (i + 1) * int(size / resolution) + j
            top_right = (i + 1) * int(size / resolution) + j + 1

            # Create two triangles for the current square
            faces.append([bottom_left, bottom_right, top_left])
            faces.append([top_left, bottom_right, top_right])

    # Write to OBJ file
    with open(filename, 'w') as f:
        for vertex in vertices:
            f.write(f"v {vertex[0]} {vertex[1]} {vertex[2]}\n")
        for face in faces:
            f.write(f"f {face[0]+1} {face[1]+1} {face[2]+1}\n")


import math

def gaussian2(x, y, sigma, height):
```

```
    """Return the height of the shape at position (x, y) using a Gaussian function."""
    return height * math.exp(-((x**2 + y**2) / (2 * sigma**2)))

def generate_gaussian_pyramid2(filename, size=10, resolution=0.5, sigma=3, height=5):
    vertices = []
    faces = []

    # Generate vertices using the Gaussian function for the pyramid
    for i in range(int(size / resolution)):
        for j in range(int(size / resolution)):
            x = -size/2 + i * resolution
            y = -size/2 + j * resolution
            z = gaussian2(x, y, sigma, height)
            vertices.append([x, y, z])

    # Generate vertices for the flat base
    for i in range(int(size / resolution)):
        for j in range(int(size / resolution)):
            x = -size/2 + i * resolution
            y = -size/2 + j * resolution
            z = 0
            vertices.append([x, y, z])

    # Generate faces by connecting vertices
    for i in range(int(size / resolution) - 1):
        for j in range(int(size / resolution) - 1):
            # Calculate indices for the current square on the pyramid
            bottom_left = i * int(size / resolution) + j
            bottom_right = i * int(size / resolution) + j + 1
            top_left = (i + 1) * int(size / resolution) + j
            top_right = (i + 1) * int(size / resolution) + j + 1

            # Create two triangles for the current square on the pyramid
            faces.append([bottom_left, bottom_right, top_left])
            faces.append([top_left, bottom_right, top_right])

            # Calculate indices for the current square on the base
            base_offset = int(size / resolution) * int(size / resolution)
            bottom_left_base = base_offset + bottom_left
            bottom_right_base = base_offset + bottom_right
            top_left_base = base_offset + top_left
            top_right_base = base_offset + top_right

            # Create two triangles for the current square on the base
            faces.append([bottom_left_base, top_left_base, bottom_right_base])
            faces.append([top_left_base, top_right_base, bottom_right_base])

    # Write to OBJ file
    with open(filename, 'w') as f:
        for vertex in vertices:
            f.write(f"v {vertex[0]} {vertex[1]} {vertex[2]}\n")
```

```
        for face in faces:
            f.write(f"f {face[0]+1} {face[1]+1} {face[2]+1}\n")


def generate_gaussian_pyramid3(filename, size=10, resolution=0.5, sigma=3, height=5,
noise_scale=0.5, noise_factor=1.0):
    """
    pyramid with noise
    """
    vertices = []
    faces = []

    # Generate vertices using the Gaussian function for the pyramid
    for i in range(int(size / resolution)):
        for j in range(int(size / resolution)):
            x = -size/2 + i * resolution
            y = -size/2 + j * resolution
            z = gaussian2(x, y, sigma, height)

            # Add Perlin noise to the z-coordinate
            z += pnoise2(x * noise_scale, y * noise_scale) * noise_factor

            vertices.append([x, y, z])

 # Generate vertices for the flat base
    for i in range(int(size / resolution)):
        for j in range(int(size / resolution)):
            x = -size/2 + i * resolution
            y = -size/2 + j * resolution
            z = 0
            vertices.append([x, y, z])

    # Generate faces by connecting vertices
    for i in range(int(size / resolution) - 1):
        for j in range(int(size / resolution) - 1):
            # Calculate indices for the current square on the pyramid
            bottom_left = i * int(size / resolution) + j
            bottom_right = i * int(size / resolution) + j + 1
            top_left = (i + 1) * int(size / resolution) + j
            top_right = (i + 1) * int(size / resolution) + j + 1

            # Create two triangles for the current square on the pyramid
            faces.append([bottom_left, bottom_right, top_left])
            faces.append([top_left, bottom_right, top_right])

            # Calculate indices for the current square on the base
            base_offset = int(size / resolution) * int(size / resolution)
            bottom_left_base = base_offset + bottom_left
            bottom_right_base = base_offset + bottom_right
            top_left_base = base_offset + top_left
            top_right_base = base_offset + top_right
```

```
            # Create two triangles for the current square on the base
            faces.append([bottom_left_base, top_left_base, bottom_right_base])
            faces.append([top_left_base, top_right_base, bottom_right_base])

    # Write to OBJ file
    with open(filename, 'w') as f:
        for vertex in vertices:
            f.write(f"v {vertex[0]} {vertex[1]} {vertex[2]}\n")
        for face in faces:
            f.write(f"f {face[0]+1} {face[1]+1} {face[2]+1}\n")



def generate_gaussian_pyramid4(filename, size=10, resolution=0.5, sigma=3, height=5,
noise_scale=0.5, noise_factor=1.0):
    vertices = []
    faces = []

    # Generate vertices using the Gaussian function for the top surface of the pyramid
    for i in range(int(size / resolution)):
        for j in range(int(size / resolution)):
            x = -size/2 + i * resolution
            y = -size/2 + j * resolution
            z = gaussian2(x, y, sigma, height)

            # Add Perlin noise to the z-coordinate
            z += pnoise2(x * noise_scale, y * noise_scale) * noise_factor

            vertices.append([x, y, z])

    # Generate vertices for the bottom surface (flat base) of the pyramid
    for i in range(int(size / resolution)):
        for j in range(int(size / resolution)):
            x = -size/2 + i * resolution
            y = -size/2 + j * resolution
            z = 0
            vertices.append([x, y, z])

    # Generate faces for the top surface
    for i in range(int(size / resolution) - 1):
        for j in range(int(size / resolution) - 1):
            bottom_left = i * int(size / resolution) + j
            bottom_right = i * int(size / resolution) + j + 1
            top_left = (i + 1) * int(size / resolution) + j
            top_right = (i + 1) * int(size / resolution) + j + 1

            faces.append([bottom_left, bottom_right, top_left])
            faces.append([top_left, bottom_right, top_right])
```

```
    # Generate faces for the bottom surface
    base_offset = int(size / resolution) * int(size / resolution)
    for i in range(int(size / resolution) - 1):
        for j in range(int(size / resolution) - 1):
            bottom_left_base = base_offset + i * int(size / resolution) + j
            bottom_right_base = base_offset + i * int(size / resolution) + j + 1
            top_left_base = base_offset + (i + 1) * int(size / resolution) + j
            top_right_base = base_offset + (i + 1) * int(size / resolution) + j + 1

            faces.append([bottom_left_base, top_left_base, bottom_right_base])
            faces.append([top_left_base, top_right_base, bottom_right_base])

    # Generate faces for the sides
    for i in range(int(size / resolution) - 1):
        for j in range(int(size / resolution)):
            top = i * int(size / resolution) + j
            bottom = base_offset + i * int(size / resolution) + j
            top_next = (i + 1) * int(size / resolution) + j
            bottom_next = base_offset + (i + 1) * int(size / resolution) + j

            faces.append([top, bottom, top_next])
            faces.append([top_next, bottom, bottom_next])

    # Write to OBJ file
    with open(filename, 'w') as f:
        for vertex in vertices:
            f.write(f"v {vertex[0]} {vertex[1]} {vertex[2]}\n")
        for face in faces:
            f.write(f"f {face[0]+1} {face[1]+1} {face[2]+1}\n")

# Generate the OBJ file
generate_gaussian_pyramid4("./shapes/gaussian_pyramid.obj")


# Generate the OBJ file
# generate_gaussian_pyramid3("./shapes/gaussian_pyramid.obj")


# Generate the OBJ file
# generate_gaussian_pyramid("./shapes/gaussian_pyramid.obj")


# make_pyramid('./shapes/pyramid.obj')
# make_rocky_moutain('./shapes/mountain_with_cubes.obj')

# make_pyramid('mountain.obj')
# make_rocky_moutain('./shapes/mountain_with_cubes.obj')
```

offline_from_csv.py

File: offline_from_csv.py

Highlighted lines indicate differences relative to the starter codebase.

```
import os
import genome
import sys
import creature
import pybullet as p
import time
import random
import numpy as np

## ... usual starter code to create a sim and floor
def main(csv_file):
    assert os.path.exists(csv_file), "Tried to load " + csv_file + " but it does not exists"

    p.connect(p.DIRECT)
    p.setPhysicsEngineParameter(enableFileCaching=0)
    p.configureDebugVisualizer(p.COV_ENABLE_GUI, 0)
    plane_shape = p.createCollisionShape(p.GEOM_PLANE)
    floor = p.createMultiBody(plane_shape, plane_shape)
    p.setGravity(0, 0, -10)
#   p.setRealTimeSimulation(1)


    # generate a random creature
    cr = creature.Creature(gene_count=1)
    dna = genome.Genome.from_csv(csv_file)
    cr.update_dna(dna)
    # save it to XML
    with open('test.urdf', 'w') as f:
        f.write(cr.to_xml())
    # load it into the sim
    rob1 = p.loadURDF('test.urdf')
    # air drop it
    p.resetBasePositionAndOrientation(rob1, [0, 0, 2.5], [0, 0, 0, 1])

    start_pos, orn = p.getBasePositionAndOrientation(rob1)

    # iterate
    elapsed_time = 0
    wait_time = 1.0/240 # seconds
    total_time = 30 # seconds
    step = 0
    while True:
        p.stepSimulation()
```

```
        step += 1
        if step % 24 == 0:
            motors = cr.get_motors()
            assert len(motors) == p.getNumJoints(rob1), "Something went wrong"
            for jid in range(p.getNumJoints(rob1)):
                mode = p.VELOCITY_CONTROL
                vel = motors[jid].get_output()
                p.setJointMotorControl2(rob1,
                            jid,
                            controlMode=mode,
                            targetVelocity=vel)
            new_pos, orn = p.getBasePositionAndOrientation(rob1)
            #print(new_pos)
            dist_moved = np.linalg.norm(np.asarray(start_pos) - np.asarray(new_pos))
            print(dist_moved)
        #time.sleep(wait_time)
        elapsed_time += wait_time
        if elapsed_time > total_time:
            break

    print("TOTAL DISTANCE MOVED:", dist_moved)



if __name__ == "__main__":
    assert len(sys.argv) == 2, "Usage: python playback_test.py csv_filename"
    main(sys.argv[1])
```

realtime_from_csv.py

File: realtime_from_csv.py

Highlighted lines indicate differences relative to the starter codebase.

```
import os
import genome
import sys
import creature
import pybullet as p
import time
import random
import numpy as np

## ... usual starter code to create a sim and floor
def main(csv_file):
    assert os.path.exists(csv_file), "Tried to load " + csv_file + " but it does not exists"

    p.connect(p.GUI)
    p.setPhysicsEngineParameter(enableFileCaching=0)
    p.configureDebugVisualizer(p.COV_ENABLE_GUI, 0)
    plane_shape = p.createCollisionShape(p.GEOM_PLANE)
    floor = p.createMultiBody(plane_shape, plane_shape)
    p.setGravity(0, 0, -10)
#   p.setRealTimeSimulation(1)


    # generate a random creature
    cr = creature.Creature(gene_count=1)
    dna = genome.Genome.from_csv(csv_file)
    cr.update_dna(dna)
    # save it to XML
    with open('test.urdf', 'w') as f:
        f.write(cr.to_xml())
    # load it into the sim
    rob1 = p.loadURDF('test.urdf')
    # air drop it
    p.resetBasePositionAndOrientation(rob1, [0, 0, 2.5], [0, 0, 0, 1])
    start_pos, orn = p.getBasePositionAndOrientation(rob1)

    # iterate
    elapsed_time = 0
    wait_time = 1.0/240 # seconds
    total_time = 30 # seconds
    step = 0
    while True:
        p.stepSimulation()
        step += 1
```

```
        if step % 24 == 0:
            motors = cr.get_motors()
            assert len(motors) == p.getNumJoints(rob1), "Something went wrong"
            for jid in range(p.getNumJoints(rob1)):
                mode = p.VELOCITY_CONTROL
                vel = motors[jid].get_output()
                p.setJointMotorControl2(rob1,
                            jid,
                            controlMode=mode,
                            targetVelocity=vel)
            new_pos, orn = p.getBasePositionAndOrientation(rob1)
            #print(new_pos)
            dist_moved = np.linalg.norm(np.asarray(start_pos) - np.asarray(new_pos))
            print(dist_moved)
        time.sleep(wait_time)
        elapsed_time += wait_time
        if elapsed_time > total_time:
            break

    print("TOTAL DISTANCE MOVED:", dist_moved)



if __name__ == "__main__":
    assert len(sys.argv) == 2, "Usage: python playback_test.py csv_filename"
    main(sys.argv[1])
```

motor_test.py

File: motor_test.py

Highlighted lines indicate differences relative to the starter codebase.

```
import genome
import creature
import pybullet as p
import time
import random
import numpy as np

## ... usual starter code to create a sim and floor
p.connect(p.GUI)
p.setPhysicsEngineParameter(enableFileCaching=0)
p.configureDebugVisualizer(p.COV_ENABLE_GUI, 0)
plane_shape = p.createCollisionShape(p.GEOM_PLANE)
floor = p.createMultiBody(plane_shape, plane_shape)
p.setGravity(0, 0, -10)
# commenting out for mac compatibility
#p.setRealTimeSimulation(1)

# generate a random creature
cr = creature.Creature(gene_count=3)
# save it to XML
with open('test.urdf', 'w') as f:
    f.write(cr.to_xml())
# load it into the sim
rob1 = p.loadURDF('test.urdf')
start_pos, orn = p.getBasePositionAndOrientation(rob1)

# iterate
elapsed_time = 0
wait_time = 1.0 / 240 # seconds
total_time = 5 # seconds
step = 0
dist_moved = 0
while True:
    p.stepSimulation()
    step += 1
    if step % 120 == 0:
        motors = cr.get_motors()
        assert len(motors) == p.getNumJoints(rob1), "Something went wrong"
        for jid in range(p.getNumJoints(rob1)):
            mode = p.VELOCITY_CONTROL
            vel = motors[jid].get_output()
            p.setJointMotorControl2(rob1,
                        jid,
```

```
                    controlMode=mode,
                    targetVelocity=vel)
        new_pos, orn = p.getBasePositionAndOrientation(rob1)
        #print(new_pos)
        dist_moved = np.linalg.norm(np.asarray(start_pos) - np.asarray(new_pos))
    print(dist_moved)
    time.sleep(wait_time)
    elapsed_time += wait_time
    if elapsed_time > total_time:
        break

print("TOTAL DISTANCE MOVED:", dist_moved)
```

starter.py

File: starter.py

Highlighted lines indicate differences relative to the starter codebase.

```
import pybullet as p
import pybullet_data as pd

p.connect(p.GUI)
p.setPhysicsEngineParameter(enableFileCaching=0)
p.configureDebugVisualizer(p.COV_ENABLE_GUI, 0)
plane_shape = p.createCollisionShape(p.GEOM_PLANE)
floor = p.createMultiBody(plane_shape, plane_shape)
p.setGravity(0, 0, -10)
```

test_ga_no_threads.py

File: test_ga_no_threads.py

Highlighted lines indicate differences relative to the starter codebase.

```
# If you on a Windows machine with any Python version
# or an M1 mac with any Python version
# or an Intel Mac with Python > 3.7
# the multi-threaded version does not work
# so instead, you can use this version.

import unittest
import population
import simulation
import genome
import creature
import numpy as np

class TestGA(unittest.TestCase):
    def testBasicGA(self):
        pop = population.Population(pop_size=10,
                                    gene_count=3)
        #sim = simulation.ThreadedSim(pool_size=1)
        sim = simulation.Simulation()

        for iteration in range(1000):
            # this is a non-threaded version
            # where we just call run_creature instead
            # of eval_population
            for cr in pop.creatures:
                sim.run_creature(cr, 2400)
            #sim.eval_population(pop, 2400)

            # new fitness extraction in case of poor creature evaluation to avoid crashes.
            fits = [getattr(cr, "fitness", 0.0) for cr in pop.creatures]

            links = [len(cr.get_expanded_links())
                    for cr in pop.creatures]
            print(iteration,
                "best fitness:", np.round(np.max(fits), 3),
                "mean:", np.round(np.mean(fits), 3),
                "mean links", np.round(np.mean(links)),
                "max links", np.round(np.max(links)))

            fit_map = population.Population.get_fitness_map(fits)
            new_creatures = []
            for i in range(len(pop.creatures)):
                p1_ind = population.Population.select_parent(fit_map)
```

```
            p2_ind = population.Population.select_parent(fit_map)
            p1 = pop.creatures[p1_ind]
            p2 = pop.creatures[p2_ind]
            # now we have the parents!
            dna = genome.Genome.crossover(p1.dna, p2.dna)
            dna = genome.Genome.point_mutate(dna, rate=0.1, amount=0.25)
            dna = genome.Genome.shrink_mutate(dna, rate=0.25)
            dna = genome.Genome.grow_mutate(dna, rate=0.1)
            cr = creature.Creature(1)
            cr.update_dna(dna)
            new_creatures.append(cr)
        # elitism
        #updated to be fitness-based
        max_fit = np.max(fits)
        best_index = int(np.argmax(fits))
        best_cr = pop.creatures[best_index]

        new_cr = creature.Creature(1)
        new_cr.update_dna(best_cr.dna)
        new_creatures[0] = new_cr

        filename = "elite_" + str(iteration) + ".csv"
        genome.Genome.to_csv(best_cr.dna, filename)


        pop.creatures = new_creatures

    self.assertNotEqual(fits[0], 0)

unittest.main()
```

test_ga.py

File: test_ga.py

Highlighted lines indicate differences relative to the starter codebase.

```
# If you on a Windows machine with any Python version
# or an M1 mac with any Python version
# or an Intel Mac with Python > 3.7
# this multi-threaded version does not work
# please use test_ga_single_thread.py on those setups

import unittest
import population
import simulation
import genome
import creature
import numpy as np

class TestGA(unittest.TestCase):
    def testBasicGA(self):
        pop = population.Population(pop_size=10,
                                    gene_count=3)
        sim = simulation.ThreadedSim(pool_size=1)
        #sim = simulation.Simulation()

        for iteration in range(1000):
            sim.eval_population(pop, 2400)
            fits = [cr.get_distance_travelled()
                    for cr in pop.creatures]
            links = [len(cr.get_expanded_links())
                    for cr in pop.creatures]
            print(iteration, "fittest:", np.round(np.max(fits), 3),
                  "mean:", np.round(np.mean(fits), 3), "mean links", np.round(np.mean(links)), "max links",
np.round(np.max(links)))
            fit_map = population.Population.get_fitness_map(fits)
            new_creatures = []
            for i in range(len(pop.creatures)):
                p1_ind = population.Population.select_parent(fit_map)
                p2_ind = population.Population.select_parent(fit_map)
                p1 = pop.creatures[p1_ind]
                p2 = pop.creatures[p2_ind]
                # now we have the parents!
                dna = genome.Genome.crossover(p1.dna, p2.dna)
                dna = genome.Genome.point_mutate(dna, rate=0.1, amount=0.25)
                dna = genome.Genome.shrink_mutate(dna, rate=0.25)
                dna = genome.Genome.grow_mutate(dna, rate=0.1)
                cr = creature.Creature(1)
                cr.update_dna(dna)
```

```
            new_creatures.append(cr)
        # elitism
        max_fit = np.max(fits)
        for cr in pop.creatures:
            if cr.get_distance_travelled() == max_fit:
                new_cr = creature.Creature(1)
                new_cr.update_dna(cr.dna)
                new_creatures[0] = new_cr
                filename = "elite_"+str(iteration)+".csv"
                genome.Genome.to_csv(cr.dna, filename)
                break

        pop.creatures = new_creatures

    self.assertNotEqual(fits[0], 0)

unittest.main()
```

test_population.py

File: test_population.py

Highlighted lines indicate differences relative to the starter codebase.

```
import unittest
import population
import numpy as np

class TestPop(unittest.TestCase):
    ## check for a parent id in the range 0-2
    def testSelPar(self):
        fits = [2.5, 1.2, 3.4]
        fitmap = population.Population.get_fitness_map(fits)
        pid = population.Population.select_parent(fitmap)
        self.assertLess(pid, 3)

    ## parent id should be 1 as the first fitness is zero
    ## second is 1000 and third is 0.1 , so second should
    ## almost always be selected
    def testSelPar2(self):
        fits = [0, 1000, 0.1]
        fitmap = population.Population.get_fitness_map(fits)
        pid = population.Population.select_parent(fitmap)
        self.assertEqual(pid, 1)

unittest.main()
```

test_genome.py

File: test_genome.py

Highlighted lines indicate differences relative to the starter codebase.

```
import unittest
import genome
import numpy as np
import os

class GenomeTest (unittest.TestCase):
    def testClassExists(self):
        self.assertIsNotNone(genome.Genome)


    def testClassExists(self):
        self.assertIsNotNone(genome.Genome)

    def testRandomGene(self):
        self.assertIsNotNone(genome.Genome.get_random_gene)

    def testRandomGeneNotNone(self):
        self.assertIsNotNone(genome.Genome.get_random_gene(5))

    def testRandomGeneHasValues(self):
        gene = genome.Genome.get_random_gene(5)
        self.assertIsNotNone(gene[0])

    def testRandomGeneLength(self):
        gene = genome.Genome.get_random_gene(20)
        self.assertEqual(len(gene), 20)

    def testRandGeneIsNumpyArrays(self):
        gene = genome.Genome.get_random_gene(20)
        self.assertEqual(type(gene), np.ndarray)

    def testRandomGenomeExists(self):
        data = genome.Genome.get_random_genome(20, 5)
        self.assertIsNotNone(data)

    def testGeneSpecExist(self):
        spec = genome.Genome.get_gene_spec()
        self.assertIsNotNone(spec)

    def testGeneSpecHasLinkLength(self):
        spec = genome.Genome.get_gene_spec()
        self.assertIsNotNone(spec['link-length'])
```

```
def testGeneSpecHasLinkLength(self):
    spec = genome.Genome.get_gene_spec()
    self.assertIsNotNone(spec['link-length']["ind"])

def testGeneSpecScale(self):
    spec = genome.Genome.get_gene_spec()
    gene = genome.Genome.get_random_gene(20)
    self.assertGreater(gene[spec["link-length"]["ind"]], 0)


def testGeneToGeneDict(self):
    spec = genome.Genome.get_gene_spec()
    gene = genome.Genome.get_random_gene(len(spec))
    gene_dict = genome.Genome.get_gene_dict(gene, spec)
    self.assertIn("link-recurrence", gene_dict)

def testGenomeToDict(self):
    spec = genome.Genome.get_gene_spec()
    dna = genome.Genome.get_random_genome(len(spec), 3)
    genome_dicts = genome.Genome.get_genome_dicts(dna, spec)
    self.assertEqual(len(genome_dicts), 3)

def testFlatLinks(self):
    links = [
      genome.URDFLink(name="A", parent_name=None, recur=1),
      genome.URDFLink(name="B", parent_name="A", recur=2),
      genome.URDFLink(name="C", parent_name="B", recur=2)
    ]
    self.assertIsNotNone(links)

def testExpandLinks(self):
    links = [
        genome.URDFLink(name="A", parent_name="None", recur=1),
        genome.URDFLink(name="B", parent_name="A", recur=1),
        genome.URDFLink(name="C", parent_name="B", recur=2),
        genome.URDFLink(name="D", parent_name="C", recur=1),
    ]
    exp_links = [links[0]]
    genome.Genome.expandLinks(links[0], links[0].name, links, exp_links)
    self.assertEqual(len(exp_links), 6)

def testCrossover(self):
    g1 = [[1], [2], [3]]
    g2 = [[4], [5], [6]]
    for i in range(10):
        g3 = genome.Genome.crossover(g1, g2)
        self.assertGreater(len(g3), 0)

def test_point(self):
    g1 = np.array([[1.0], [2.0], [3.0]])
```

```
    g2 = genome.Genome.point_mutate(g1, rate=1, amount=0.25)
    self.assertFalse(np.array_equal(g1, g2))

def test_point_range(self):
    g1 = np.array([[1.0], [0.0], [1.0], [0.0]])
    for i in range(100):
        g2 = genome.Genome.point_mutate(g1, rate=1, amount=0.25)
        self.assertLessEqual(np.max(g2), 1.0)
        self.assertGreaterEqual(np.min(g2), 0.0)

def test_shrink(self):
    g1 = np.array([[1.0], [2.0]])
    g2 = genome.Genome.shrink_mutate(g1, rate=1.0)
    # should def. shrink as rate = 1
    self.assertEqual(len(g2), 1)

def test_shrink2(self):
    g1 = np.array([[1.0], [2.0]])
    g2 = genome.Genome.shrink_mutate(g1, rate=0.0)
    # should not shrink as rate = 0
    self.assertEqual(len(g2), 2)

def test_shrink3(self):
    g1 = np.array([[1.0]])
    g2 = genome.Genome.shrink_mutate(g1, rate=1.0)
    # should not shrink if already len 1
    self.assertEqual(len(g2), 1)

def test_grow1(self):
    g1 = np.array([[1.0], [2.0]])
    g2 = genome.Genome.grow_mutate(g1, rate=1)
    self.assertGreater(len(g2), len(g1))

def test_grow2(self):
    g1 = np.array([[1.0], [2.0]])
    g2 = genome.Genome.grow_mutate(g1, rate=0)
    self.assertEqual(len(g2), len(g1))

def test_tocsv(self):
    g1 = [[1,2,3]]
    genome.Genome.to_csv(g1, 'test.csv')
    self.assertTrue(os.path.exists('test.csv'))

def test_tocsv_content(self):
    g1 = [[1,2,3]]
    genome.Genome.to_csv(g1, 'test.csv')
    expect = "1,2,3,\n"
    with open('test.csv') as f:
        csv_str = f.read()
    self.assertEqual(csv_str, expect)
```

```
    def test_tocsv_content2(self):
        g1 = [[1,2,3], [4,5,6]]
        genome.Genome.to_csv(g1, 'test.csv')
        expect = "1,2,3,\n4,5,6,\n"
        with open('test.csv') as f:
            csv_str = f.read()
        self.assertEqual(csv_str, expect)

    def test_from_csv(self):
        g1 = [[1,2,3]]
        genome.Genome.to_csv(g1, 'test.csv')
        g2 = genome.Genome.from_csv('test.csv')
        print(g1, g2)
        self.assertTrue(np.array_equal(g1, g2))

    def test_from_csv2(self):
        g1 = [[1,2,3], [4,5,6]]
        genome.Genome.to_csv(g1, 'test.csv')
        g2 = genome.Genome.from_csv('test.csv')
        print(g1, g2)
        self.assertTrue(np.array_equal(g1, g2))

unittest.main()
```

test_simulation.py

File: test_simulation.py

Highlighted lines indicate differences relative to the starter codebase.

```
import unittest
import simulation
import creature
import population

class TestSim(unittest.TestCase):
    def testSimExists(self):
        sim = simulation.Simulation()
        self.assertIsNotNone(sim)

    def testSimId(self):
        sim = simulation.Simulation()
        self.assertIsNotNone(sim.physicsClientId)

    def testRun(self):
        sim = simulation.Simulation()
        self.assertIsNotNone(sim.run_creature)

    def testPos(self):
        sim = simulation.Simulation()
        cr = creature.Creature(gene_count = 3)
        sim.run_creature(cr)
        self.assertNotEqual(cr.start_position, cr.last_position)

    def testPop(self):
        pop = population.Population(pop_size=5, gene_count=3)
        sim = simulation.Simulation()
        for cr in pop.creatures:
            sim.run_creature(cr)
        dists = [cr.get_distance_travelled() for cr in pop.creatures]
        print(dists)
        self.assertIsNotNone(dists)

## uncomment this to test the
## multi-threaded sim
#    def testProc(self):
#        pop = population.Population(pop_size=20, gene_count=3)
#        tsim = simulation.ThreadedSim(pool_size=8)
#        tsim.eval_population(pop, 2400)
#        dists = [cr.get_distance_travelled() for cr in pop.creatures]
#        print(dists)
#        self.assertIsNotNone(dists)
```

```
    def testProcNoThread(self):
        pop = population.Population(pop_size=20, gene_count=3)
        sim = simulation.Simulation()
        sim.eval_population(pop, 2400)
        dists = [cr.get_distance_travelled() for cr in pop.creatures]
        print(dists)
        self.assertIsNotNone(dists)

unittest.main()
```

test_creature.py

File: test_creature.py

Highlighted lines indicate differences relative to the starter codebase.

```
import unittest
import creature
import pybullet as p

class TestCreature(unittest.TestCase):
    def testCreatExists(self):
        self.assertIsNotNone(creature.Creature)

    def testCreatureGetFlatLinks(self):
        c = creature.Creature(gene_count=4)
        links = c.get_flat_links()
        self.assertEqual(len(links), 4)

    def testExpLinks(self):
        c = creature.Creature(gene_count=25)
        links = c.get_flat_links()
        exp_links = c.get_expanded_links()
        self.assertGreaterEqual(len(exp_links), len(links))

    def testToXMLNotNone(self):
        c = creature.Creature(gene_count=2)
        xml_str = c.to_xml()
        self.assertIsNotNone(xml_str)

    def testLoadXML(self):
        c = creature.Creature(gene_count=20)
        xml_str = c.to_xml()
        with open('test.urdf', 'w') as f:
            f.write(xml_str)
        p.connect(p.DIRECT)
        cid = p.loadURDF('test.urdf')
        self.assertIsNotNone(cid)


    def testMotor(self):
        m = creature.Motor(0.1, 0.5, 0.5)
        self.assertIsNotNone(m)

    def testMotorVal(self):
        m = creature.Motor(0.1, 0.5, 0.5)
        self.assertEqual(m.get_output(), 1)

    def testMotorVal2(self):
```

```
        m = creature.Motor(0.6, 0.5, 0.5)
        m.get_output()
        m.get_output()
        self.assertGreater(m.get_output(), 0)

    def testDist(self):
        c = creature.Creature(3)
        c.update_position((0, 0, 0))
        d1 = c.get_distance_travelled()
        c.update_position((1, 1, 1))
        d2 = c.get_distance_travelled()
        self.assertGreater(d2, d1)


unittest.main()
```